

Практическая теория типов

Лекция 12: Neuro-symbolic AI & Type Theory: Trends 2023–2025

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

«Нейросети дают интуицию, типы дают гарантии»

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические кейсы и направления развития

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические кейсы и направления развития

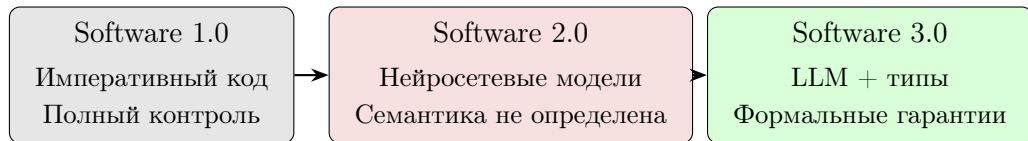
Место лекции в структуре курса

Пройденный материал:

- Лекции 1–2: λ -исчисление, редукции
- Лекция 3: Просто типизированное λ -исчисление
- Лекции 4–5: Алгебраические типы данных
- Лекция 6: System F, полиморфизм
- Лекция 7: Соответствие Карри–Ховарда
- Лекция 8: Субструктурные и сессионные типы
- Лекция 9: $\lambda\omega$, λP , зависимые типы
- Лекции 10–11: CIC, Coq/Lean, основы HoTT

Тема лекции: применение теории типов в контексте LLM и информационной безопасности.

Эволюция парадигм разработки ПО



В парадигме Software 3.0 типовые системы и пруверы выступают в роли верификаторов для артефактов, генерируемых LLM.

Актуальность для информационной безопасности

Новые поверхности атаки:

- AI-ассистенты для разработки (Copilot, Cursor, Devin) в критичных репозиториях
- LLM-агенты с доступом к API и инфраструктуре
- Смарт-контракты и криптографические протоколы, генерируемые ИИ
- Финансовые системы с AI-компонентами

Теория типов и формальные методы представляют собой инструментарий для обеспечения контролируемости таких систем.

С ростом возможностей AI возрастает потребность в формальных гарантиях.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические кейсы и направления развития

Ограничения нейросетевых подходов

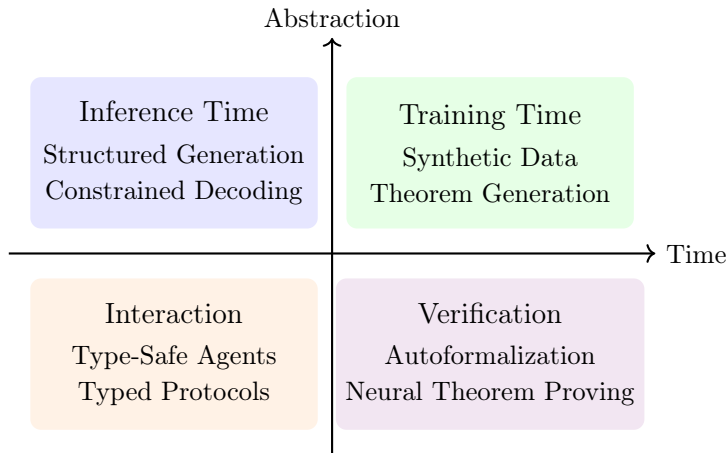
Известные проблемы LLM:

- Галлюцинации: генерация синтаксически корректного, но семантически неверного вывода
- Отсутствие формального reasoning: статистическая, а не логическая природа вывода
- По данным исследований, 25–40% AI-генерируемого кода содержит уязвимости (Pearce et al., 2022; Perry et al., 2023)
- Prompt injection как новый класс атак

Neural $\neq$ Reliable

Вывод: использование LLM в критичных системах требует дополнительной валидации.

Карта исследовательских направлений



Каждое направление опирается на изученные концепции:

Концепция	Лекция	Применение в AI
ADT (суммы, произведения)	5	JSON Schema, действия агентов
Exhaustiveness checking	5	Полнота обработки действий
Curry–Howard	7	LLM ищет терм для типа
Сессионные типы	8	Протоколы агент $\leftrightarrow$ API
Линейные типы	8	Ресурсы, токены, транзакции
Σ -типы, refinement	9	Constraints, PCC
CIC, Lean/Coq	10–11	Neural theorem proving

Общий принцип: LLM генерирует термы, типовая система верифицирует.

Эта лекция — не «приложение», а кульминация курса.

Нейронная компонента:

- LLM (GPT, Claude, Llama)
- Reinforcement Learning
- Neural search (MCTS + NN)

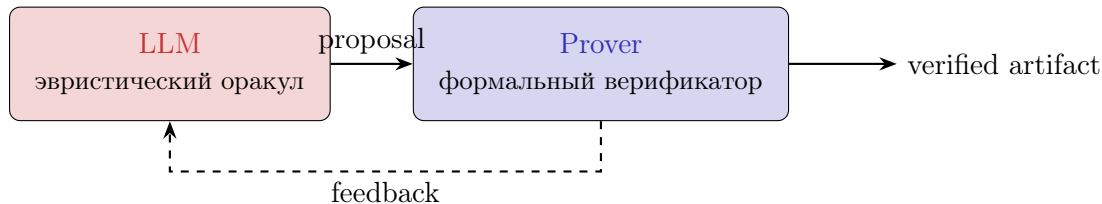
Символическая компонента:

- Формальная логика, типы
- Proof assistants
- SAT/SMT solvers

Ключевые направления 2023–2025:

- 1 Neural Theorem Proving (AlphaProof, Baldur, LeanDojo)
- 2 Autoformalization (NL $\rightarrow$ Formal Spec)
- 3 Typed Agents (Function calling, MCP)
- 4 Proof-Carrying Artifacts

Архитектурный паттерн



Паттерн: Neural Proposal $\rightarrow$ Symbolic Verification

- 1 LLM генерирует кандидата (код, доказательство, план)
- 2 Prover верифицирует; при ошибке — итеративное уточнение
- 3 При успехе — формально корректный артефакт

- ① Structured Generation — constrained decoding, грамматики, типы как спецификации
- ② Neural Theorem Proving — AlphaProof, autoformalization, proof repair
- ③ Typed Agents — function calling, MCP, сессионные типы
- ④ Code Synthesis + Proof-Carrying — feedback loops, PCC
- ⑤ Практические кейсы и направления развития

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding**
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические кейсы и направления развития

Проблема неструктурированного вывода

$$P(\text{token}_t \mid \text{context}) = \text{softmax}(\text{logits})$$

Языковая модель не учитывает синтаксические ограничения целевого формата.

Ожидаемый вывод:

```
{"name": "Alice", "age": 25}
```

Возможный вывод:

```
Sure! Here's the JSON:  
{"name": "Alice, "age": 25
```

Некорректный JSON приводит к ошибкам парсинга и потенциальным уязвимостям.

Типы как грамматики

Ключевое наблюдение: тип или схема определяет контекстно-свободную грамматику.

Вместо инструктирования модели через prompt используется принудительное ограничение пространства вывода.

Примеры схем:

- JSON Schema
- Pydantic models (Python)
- TypeScript interfaces
- Protocol Buffers

На каждом шаге декодирования разрешаются только токены, продолжающие валидный префикс.

Связь с ADT: схема = алгебраический тип

Вспомним лекцию 5: ADT = суммы (+) и произведения ($\times$).

JSON Schema — это в точности ADT:

"type": "object" $\cong$ произведение (record, $\times$)

"oneOf": [...] $\cong$ сумма (variant, +)

"enum": ["a" "b"] $\cong$ $1 + 1$ (конечный тип)

"required": [...] $\cong$ non-nullable (исключение None)

"items": {...} $\cong$ $\mu X. 1 + T \times X$ (список)

Принцип из лекции 5: make illegal states unrepresentable.

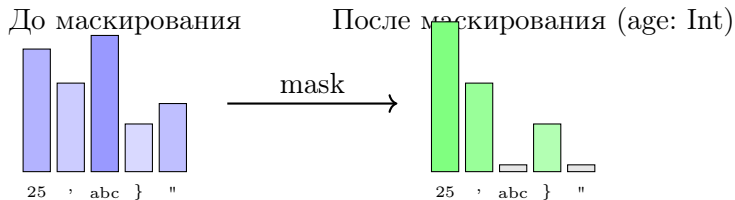
Structured generation реализует этот принцип на этапе генерации, а не парсинга.

Инструменты structured generation

- Outlines (dottxt-ai) — преобразование JSON Schema в конечный автомат
- llama.cpp grammars — BNF-грамматики с минимальными накладными расходами
- Guidance (Microsoft) — программное управление генерацией
- LMQL (ETH Zürich) — декларативный язык запросов с ограничениями
- SGLang (Berkeley) — язык структурированной генерации

Structured generation обеспечивает 100% синтаксическую корректность при накладных расходах порядка 5–10% по latency.

Механизм Logit Masking



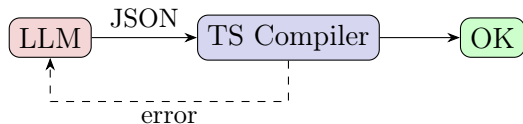
Недопустимые токены получают значение $\text{logit} = -\infty$.

Грамматика определяет проекцию на множество допустимых продолжений.

Сравнительный анализ подходов

Критерий	Unstructured	Structured
Синтаксическая корректность	~90–95%	100%
Соответствие схеме	~70–85%	100%
Накладные расходы	baseline	+5–10%
Обработка downstream	exception handling	гарантированный парсинг
Устойчивость к injection	низкая	высокая

Structured generation представляет существенное преимущество для production-систем.



Подход: использование TypeScript-интерфейсов как спецификации вывода.

- ❶ Определение интерфейса в TypeScript
- ❷ Генерация JSON моделью
- ❸ Валидация компилятором TypeScript
- ❹ При ошибке — итеративное исправление

github.com/microsoft/TypeChat

Семантические ограничения: LMQl и DSPy

LMQL — декларативные ограничения:

```
argmax "Score: [SCORE]" from "gpt-4"  
where INT(SCORE) and 0 <= INT(SCORE) <= 10
```

DSPy — программные assertions:

```
dspy.Assert(len(response.items) <= 5, "Too many items")  
dspy.Suggest(is_valid_date(response.date), "Invalid date")
```

Связь с лекцией 9 (λP): это в точности refinement types!

$$\{x : \text{Int} \mid 0 \leq x \leq 10\} \cong \Sigma x : \text{Int}. (0 \leq x) \times (x \leq 10)$$

LMQL/DSPy реализуют runtime-аппроксимацию Σ -типов из зависимой теории типов.

Применение для защиты от инъекций

Рассмотрим поле "role": "user "admin".

Попытка атаки:

```
{"role": "user\", \"admin\": true}
```

При использовании грамматики допустимы только токены "user" или "admin".
Инъекция становится невозможной на уровне генерации.

Типизация определяет ограничение канала вывода.
Defense in depth: типы на входе и на выходе.

Текущие ограничения реализуются как runtime assertions:

- $0 \leq \text{score} \leq 10$
- $\text{len}(\text{items}) < 100$
- $\text{date} \geq \text{today}$

Направление развития: зависимые типы для structured generation.

Длина массива, свойство отсортированности, инварианты структур данных — всё это может быть выражено в типах и верифицировано при декодировании.

См. Liquid Types (Rondon et al.), Refinement Types (Xi & Pfenning)

- Эффективные алгоритмы инкрементального парсинга
- Комбинирование синтаксических грамматик и семантических ограничений
- Типизированное декодирование для сложных структур (AST, планы агентов)
- Интеграция SMT solvers в цикл декодирования

Ключевые публикации:

- Willard & Louf (2023): Outlines
- Beurer-Kellner et al. (2023): LMQL
- Khattab et al. (2023): DSPy

Пример: Pydantic + Outlines

```
from pydantic import BaseModel
from typing import Literal
import outlines

class Person(BaseModel):
    name: str
    age: int
    role: Literal["user", "admin"]

model = outlines.models.transformers("mistralai/Mistral-7B")
generator = outlines.generate.json(model, Person)

person = generator("Create a user named Alice")
# {"name": "Alice", "age": 28, "role": "user"}
```

Результат гарантированно соответствует схеме.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization**
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические кейсы и направления развития

Проблема ручной формальной верификации

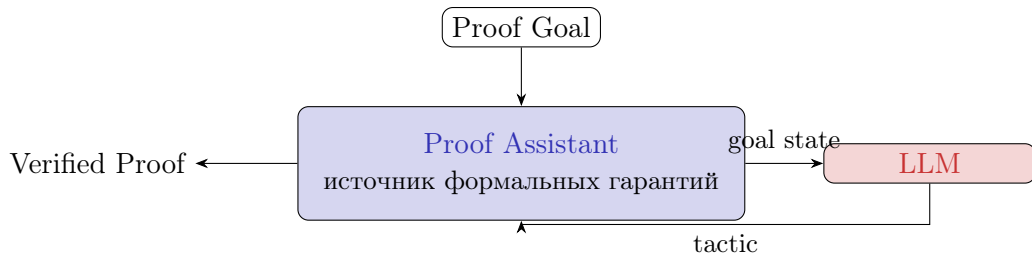
Разработка доказательств в Coq/Lean — ресурсоёмкий процесс.

- Высокий порог входа: требуются годы обучения
- Соотношение: 10–20 строк доказательства на 1 строку кода
- CompCert (верифицированный компилятор C): 6 человеко-лет
- seL4 (микроядро): ~200k строк доказательств
- Mathlib (Lean): более 1M строк

Для криптографических протоколов, смарт-контрактов, критичного ПО — нехватка специалистов по формальной верификации является узким местом.

Вопрос: возможно ли применение AI для ускорения при сохранении гарантий?

LLM как генератор тактик



Цикл: Propose $\rightarrow$ Check $\rightarrow$ Repair

LLM предлагает тактику, proof assistant верифицирует, при ошибке — обратная связь.
При успехе доказательство формально верифицировано.

Curry–Howard в действии: LLM ищет терм

Вспомним лекцию 7: соответствие Карри–Ховарда.

Теорема	$\longleftrightarrow$	Тип
Доказательство	$\longleftrightarrow$	Терм (программа)
Proof Goal $\Gamma \vdash ?M : \tau$	$\longleftrightarrow$	Type Inhabitation

Что делает LLM? Генерирует терм M для заданного типа τ .

$$\text{LLM} : \Gamma \vdash ? : \tau \longrightarrow \Gamma \vdash M : \tau$$

Prover выполняет type checking: проверяет $\Gamma \vdash M : \tau$.

Это разделение генерации и верификации — ключ к надёжности.

AlphaProof и AlphaGeometry

AlphaGeometry (DeepMind, January 2024):

- Комбинация нейросети и символического решателя
- Решено 25 из 30 задач геометрии IMO за 30 лет
- Trinh et al., Nature 2024

AlphaProof (DeepMind, July 2024):

- Lean 4 + reinforcement learning
- IMO 2024: результат на уровне серебряной медали (4 из 6 задач)
- Все решения формально верифицированы в Lean

Нейросетевая компонента обеспечивает эвристику, proof assistant — гарантии.

Lean 4 (Microsoft Research):

- Современный proof assistant с компиляцией в C
- Mathlib: более 1M строк формализованной математики, свыше 100k теорем

LeanDojo (Yang et al., NeurIPS 2023):

- Программный интерфейс LLM $\leftrightarrow$ Lean
- Benchmark: 98k теорем
- ReProver: модель с лучшими показателями на данном benchmark

Связанные проекты: Pantograph, ntp-toolkit, LeanCopilot

leanprover.github.io leandojo.org

Связь с CIC: почему Lean/Coq идеальны для NTP

Вспомним лекцию 10: Calculus of Inductive Constructions.

Почему CIC подходит для neural theorem proving:

- 1 Decidable type checking: верификация — $O(\text{poly})$, генерация — NP-hard
- 2 Зависимые типы: спецификации в типах ($\text{Vec } A\ n, \text{ sorted } xs$)
- 3 Тактики: высокоуровневый язык для построения proof terms
- 4 Kernel: маленькое доверенное ядро проверяет всё

Разделение ролей:

- LLM генерирует тактики (эвристика, паттерны)
- Kernel проверяет proof term (формальная гарантия)

Даже если LLM ошибается в 90% случаев — 10% успешных доказательств верны на 100%.

Автоматический перевод естественного языка в формальные спецификации:

Естественный язык	Формальная запись
«Сумма двух чётных чисел чётна»	$\text{even } a \wedge \text{even } b \rightarrow \text{even } (a+b)$
«Список отсортирован»	$\forall i < j. xs_i \leq xs_j$
«Пароль не хранится в открытом виде»	$\forall s. \text{encrypted}(s.\text{password})$

Мотивация: спецификации в документации (RFC и др.) формулируются на естественном языке.

Формализация обеспечивает точность и возможность автоматической верификации.

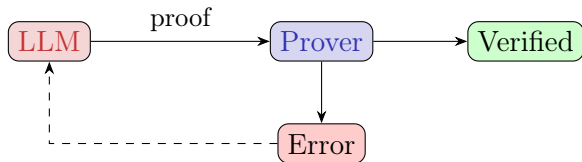
Сценарий применения:

- 1 Текстовое описание протокола (RFC, whitepaper)
- 2 $\xrightarrow{\text{autoformalize}}$ формальная модель (TLA+, Lean)
- 3 Верификация свойств безопасности
- 4 Получение доказательства или контрпримера

Области применения:

- Протоколы аутентификации (OAuth, SAML, OIDC)
- Алгоритмы консенсуса (BFT, PoS)
- Системы электронного голосования
- Политики контроля доступа

Инструменты: TLA+, Tamarin, ProVerif, Lean/Coq



Baldur (First et al., 2023): LLM генерирует доказательство, Isabelle возвращает сообщение об ошибке, LLM использует его для итеративного улучшения.

Proof obligations представляют собой идеальный сигнал обратной связи: бинарный, точный, формально определённый.

Другие системы: COPRA, LEGO-Prover, DeepSeek-Prover (2024)

Draft, Sketch, and Prove



- ① Draft: неформальное рассуждение на естественном языке
- ② Sketch: полуформальный скелет с незаполненными частями (sorry)
- ③ Prove: полноценное формальное доказательство

Jiang et al. (2023): Draft, Sketch, and Prove

Синтетические данные для обучения

Проблема: объём формальных доказательств ограничен (Mathlib: 100k vs ImageNet: 14M).

Подход: синтетические данные + self-play

- Автоматическая генерация теорем
- Попытки построения доказательств
- Успешные примеры используются для обучения

Аналогия с AlphaZero: self-play в настольных играх $\rightarrow$ self-play в пространстве доказательств.

Polu et al. (2022), Wang et al. (2023), Xin et al. (2024)

Верификация кода:

- Verus (Rust + SMT), Creusot (Rust + Why3)
- Dafny, F*, Liquid Haskell

Автоматический синтез инвариантов: LLM генерирует кандидаты, SMT solver верифицирует.

Интеграция в CI/CD:

- 1 Код $\rightarrow$ LLM генерирует спецификацию
- 2 LLM генерирует скелет доказательства
- 3 Prover верифицирует
- 4 Слияние только при успешной верификации

Пример: Lean + LLM

```
theorem add_comm (a b : Nat) : a + b = b + a := by  
  sorry -- to be filled
```

Предложение LLM:

```
theorem add_comm (a b : Nat) : a + b = b + a := by  
  induction a with  
  | zero => simp  
  | succ n ih => simp [Nat.succ_add, ih]
```

Goal state $\rightarrow$ LLM $\rightarrow$ tactic $\rightarrow$ верификация Lean $\rightarrow$ QED

Инструменты: LeanCopilot, lean-gptf, ReProver

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы**
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические кейсы и направления развития

Переход от чат-ботов к агентам

$$\text{Agent} = \text{LLM} + \text{Memory} + \text{Tools} + \text{Goal}$$

Примеры: Cursor, Devin, Perplexity, Zapier AI

Риски:

- Некорректный вызов API → повреждение данных
- Ошибочное удаление базы данных
- Неавторизованные финансовые операции
- Prompt injection → перехват управления

Агент обладает реальными полномочиями при ненадёжности LLM как источника решений.

Типы как контракты действий

```
data Action
= Search String
| Calculator Expr
| Transfer Account Account Amount
| RunSQL Query
| SendEmail EmailAddress Body
```

Агент ограничен выбором из замкнутого множества действий, определённого ADT.

Без типизации: агент может генерировать вызовы несуществующих API.

С типизацией: замкнутое множество действий, валидированные аргументы, structured generation.

ADT для агентов: связь с лекцией 5

Вспомним лекцию 5: сумма типов = закрытое множество вариантов.

$$\text{Action} = \text{Search} \times \text{String} + \text{Transfer} \times \text{Account}^2 \times \text{Amount} + \dots$$

Exhaustiveness checking (лекция 5):

- Компилятор гарантирует обработку всех вариантов
- Добавление нового действия $\Rightarrow$ ошибка компиляции в обработчиках
- Невозможно «забыть» обработать действие

Make illegal states unrepresentable:

- Transfer требует два аккаунта и сумму — по типу
- Невозможно создать Transfer без всех аргументов
- Structured generation: LLM обязан заполнить все поля

Function Calling

```
{
  "name": "get_weather",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {"type": "string"},
      "unit": {"enum": ["celsius", "fahrenheit"]}
    },
    "required": ["location"]
  }
}
```

Схема = тип, enum = sum type, required = non-nullable.

Поддержка: OpenAI, Anthropic, Google.

Gorilla и проблема hallucinated API

Gorilla (Patil et al., 2023): LLM, дообученная для вызова API.

Проблема: hallucinated APIs — генерация вызовов несуществующих функций.

Решение:

- ❶ Каталог API с типизированными схемами
- ❷ Retrieval релевантных API
- ❸ Constrained generation: только из каталога
- ❹ Валидация сигнатур

gorilla.cs.berkeley.edu

Model Context Protocol (MCP)

MCP (Anthropic, 2024) — стандарт взаимодействия LLM и инструментов.



Каждый инструмент имеет типизированную схему, вход и выход типизированы. MCP-server выступает типизированным слоем между агентом и внешним миром.

modelcontextprotocol.io

Переход от промптинга к программированию:

- LMQL: декларативные запросы к LLM
- DSPy: программирование с assertions
- LangChain: композиция вызовов
- LangGraph: графы состояний

Pipeline представляет собой граф вызовов модели с типизированным состоянием. State соответствует record type, transitions — функциям.

Сессионные типы для протоколов агентов

```
type DBAgentProtocol =  
  Hello .> Auth Credentials .> AuthResult  
    .> (Query SQL .> QueryResult)*  
    .> Bye
```

Сессионный тип определяет допустимые последовательности сообщений:

- Обязательная инициализация (Hello)
- Аутентификация перед запросами
- Корректное завершение сессии

Применения: database agent, wallet agent, admin agent.

Линейные типы из лекции 8: ресурсы агента

Вспомним лекцию 8: линейные и аффинные типы для ресурсов.

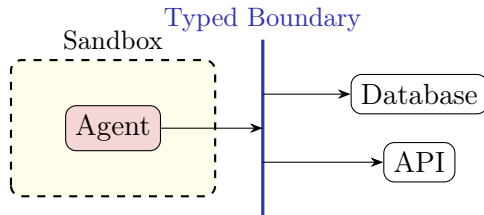
Прямая аналогия с агентами:

Файл (открыть/закрыть)	$\longleftrightarrow$	API-токен (получить/отозвать)
$\&\text{mut } T$ (эксклюзивный доступ)	$\longleftrightarrow$	Lock на ресурс
Move semantics	$\longleftrightarrow$	Передача ownership данных
Session type	$\longleftrightarrow$	Протокол агент $\leftrightarrow$ API

Гарантии из лекции 8, применимые к агентам:

- No use-after-free: агент не использует отозванный токен
- No double-free: транзакция выполняется ровно один раз
- Protocol compliance: последовательность действий соответствует спецификации

Sandboxing и типизированные границы



Агент изолирован в песочнице, взаимодействие с внешним миром — через типизированные границы.

Defense in depth: типизация входа, действий и выхода.

План агента представляет собой AST, допускающий статический анализ до выполнения.

Возможные проверки:

- Типизация: корректность всех действий
- Инварианты безопасности: отсутствие запрещённых операций
- Проверка полномочий агента
- Ограничения на ресурсы: гарантия завершения

Корректный план может рассматриваться как доказательство безопасности workflow. Применение соответствия Карри–Ховарда.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts**
- 7 Практические кейсы и направления развития

Генерация кода с помощью LLM

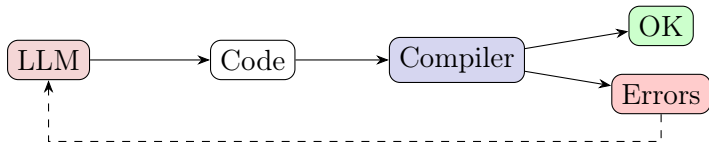
По оценкам, GitHub Copilot генерирует более 40% кода. Более 100М разработчиков используют AI-ассистенты.

Проблема:

- По данным исследований, 25–40% AI-генерируемого кода содержит дефекты и уязвимости
- Типичные уязвимости: SQL injection, path traversal, отсутствие валидации
- «Less secure than human code» (Perry et al., 2023)

Код для критичных систем требует дополнительной валидации.

Feedback Loops



LLM → код → компилятор → ошибки → итеративное исправление.

Self-repair улучшает показатели: синтаксис +100%, типы +20–30%, тесты +15–25%.

Chen et al. (2023): Self-Debugging

Property-Based Testing

QuickCheck/Hypothesis: генерация тестов на основе свойств.

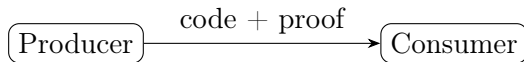
- 1 LLM генерирует код
- 2 LLM генерирует свойства (properties)
- 3 Hypothesis/QuickCheck выполняет тестирование
- 4 Контрпримеры используются как обратная связь

Properties представляют собой частичную спецификацию.

Не эквивалентно полной верификации, но превосходит unit-тестирование.

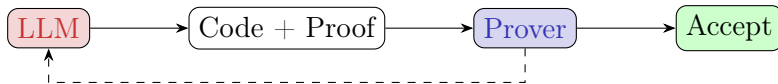
Proof-Carrying Code (PCC)

Концепция (Necula, 1997): код поставляется вместе с доказательством свойств безопасности.



Consumer выполняет только проверку доказательства — вычислительно дешёвая операция.

Типичные свойства: memory safety, type safety, отсутствие переполнений, соблюдение политик безопасности.



LLM генерирует пару (код, доказательство), prover выполняет верификацию. Код без валидного доказательства отклоняется.

Инструменты: Dafny, Verus, F*, Lean/Coq

Proof-Carrying Numbers и Data

Proof-Carrying Numbers: каждое значение сопровождается доказательством свойств.

Пример: $\text{sum} : \{n : \mathbb{N} \mid n \leq \text{limit}\}$

Proof-Carrying Data: данные с верифицируемой историей трансформаций.

Каждая операция оставляет доказательство корректности.

Применения:

- DeFi: сумма транзакции не превышает баланс
- ML pipelines: гарантии на predictions
- zk-proofs: verifiable computation

РСС через призму Σ -типов (лекция 9)

Вспомним лекцию 9: Σ -тип = зависимая пара.

$$\Sigma x : A. P(x) \cong \text{«значение } x \text{ вместе с доказательством } P(x)\text{»}$$

Proof-Carrying Code — это Σ -тип:

$$\text{PCC} = \Sigma \text{code} : \text{Program}. \text{Safe}(\text{code})$$

- Первая компонента: программа (код)
- Вторая компонента: доказательство свойства безопасности
- Невозможно создать РСС без валидного доказательства

Curry–Howard (лекция 7): доказательство $\text{Safe}(\text{code})$ — это терм.

LLM генерирует пару $(\text{code}, \text{proof})$, prover проверяет типизацию proof .

Zero-Knowledge Proofs и LLM

ZK-proofs: доказательство свойства без раскрытия данных.

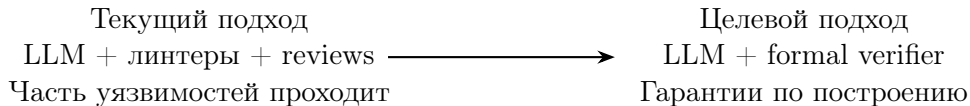
Применение LLM:

- Помощь в построении circuits
- Генерация witness
- Верификация остаётся формальной

Области применения:

- Verifiable ML inference
- Private credentials
- Blockchain bridges

От Best Effort к By Construction



Код без доказательства отклоняется. Развёртывание только верифицированного кода.
Отсутствие уязвимостей определённого класса гарантируется конструктивно.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические кейсы и направления развития

Кейс 1: AI-ассистент в банковском репозитории

Сценарий: Copilot/Cursor в кодовой базе финансовой организации.

Риски: неконтролируемые изменения, уязвимости, нарушение compliance.

Решение:

- 1 Structured generation: только типизированные патчи
- 2 Typed API boundaries: изоляция credentials
- 3 Proof-carrying patches: формальная проверка каждого PR
- 4 Audit trail: полное логирование действий AI

Кейс 2: LLM-агент для платёжных операций

Сценарий: агент с доступом к платёжному API или криптокошельку. Некорректный вызов приводит к финансовым потерям.

Применение типов:

- ADT для действий: Transfer to amt с ограничениями
- Сессионные типы: Auth $\rightarrow$ Approve $\rightarrow$ Execute
- Proof-carrying transactions
- Rate limits как типы: Amount<daily_limit>

План агента верифицируется до выполнения.

Кейс 3: Верификация протокола с применением AI

Сценарий: протокол аутентификации или консенсуса.

Роль LLM:

- 1 Autoformalization: текст $\rightarrow$ TLA+ / Lean
- 2 Предложение инвариантов
- 3 Генерация скелета доказательства
- 4 Prover обеспечивает верификацию

Примеры: OAuth, TLS, Raft/PBFT, e-voting, blockchain bridges.

Инструменты: TLA+, Tamarin, ProVerif, Lean/Coq, Alloy

Профессиональные роли

AI Safety / Alignment Engineer

Anthropic, OpenAI, DeepMind. Компенсация: \$200–400k+ (senior).

Formal Verification Engineer

Blockchain (Certora), финтех (Jane Street), аэрокосмическая отрасль.

ML Infra Security

Secure MLOps, безопасность моделей, adversarial robustness.

Cryptography / ZK + AI

zk-proofs для ML, trusted compute, гомоморфное шифрование.

На пересечении формальных методов и AI наблюдается растущий спрос на специалистов.

Рекомендуемые ресурсы

Proof Assistants:

- Lean 4 + Mathlib: leanprover.github.io
- Software Foundations: softwarefoundations.cis.upenn.edu

Structured Generation:

- Outlines: github.com/dottxt-ai/outlines
- LMQL: lmql.ai, DSPy: github.com/stanfordnlp/dspy

Neural Theorem Proving:

- LeanDojo: leandojo.org
- LeanCopilot: github.com/lean-dojo/LeanCopilot

Agents: MCP: modelcontextprotocol.io

Темы для исследовательских работ

Neuro-symbolic:

- Neuro-symbolic верификация протоколов агентов
- Autoformalization криптографического протокола (TLS/Signal)

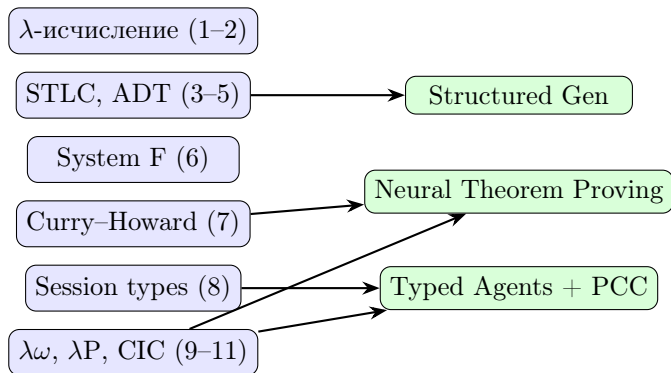
Proof-Carrying:

- RCC completions для подмножества Rust
- Верифицированный валидатор JSON Schema в Lean

Agents:

- Сессионные типы для MCP-протокола
- Type-safe agent framework с верификацией планов

Как весь курс сходится к этой лекции



Лекция 12 — не приложение, а синтез всего курса.

«Будущее не за чистым ИИ и не за чистой математикой,
а за их симбиозом.

Вы — те, кто может построить этот мост.»

- Structured generation: типизация вывода LLM
- Neural theorem proving: комбинация AI и формальных доказательств
- Typed agents: безопасные LLM-агенты
- Proof-carrying artifacts: гарантии по построению

Типы обеспечивают гарантии. Нейросети обеспечивают эвристику.